

Python

Summary Session 9



Преподаватель

Портрет

Имя Фамилия

Текущая должность

Количество лет опыта

Какой у Вас опыт - ключевые кейсы

Самые яркие проекты

Дополнительная информация по вашему усмотрению

Корпоративный e-mail

Социальные сети (по желанию)

Важно

-  Камера должна быть включена на протяжении всего занятия
-  В течение занятия вопросы задавать в чате или когда преподаватель спрашивает, есть ли у Вас вопросы
-  Вести себя уважительно и этично по отношению к остальным участникам занятия
-  Организационные вопросы по обучению решаются с кураторами, а не на тематических занятиях
-  Во время занятия будут интерактивные задания, будьте готовы включить камеру или демонстрацию экрана по просьбе преподавателя



ПОВТОРЕНИЕ ИЗУЧЕННОГО



Экспресс-опрос



Что такое множество?



Множество

Это изменяемая структура данных, которая хранит уникальные элементы в неупорядоченном виде. Множество используется для выполнения различных операций с уникальными элементами, таких как объединение, пересечение и разность.

Особенности множества



Хеширование элементов: При добавлении элемента в множество вычисляется его хеш-код с помощью функции `hash()`. Хеш-код используется для определения позиции элемента в памяти.



Уникальность элементов: Если элемент с таким же хеш-кодом уже существует, то новый элемент не добавляется, так как множество поддерживает только уникальные значения.



Неупорядоченность: Множество не сохраняет порядок добавления элементов.



Изменяемость: Множество можно изменять (добавлять и удалять элементы).

Ограничения



Только хешируемые объекты: Множество может хранить только хешируемые объекты, такие как числа, строки и кортежи.



Неупорядоченность: Порядок элементов не сохраняется, что делает невозможным доступ по индексу.

Создание множества



Создание множества с помощью фигурных скобок

```
unique_numbers = {1, 2, 3, 4, 5}
```

```
print(unique_numbers)
```

Создание пустого множества с использованием функции set()

```
empty_set = set()
```

```
print(empty_set)
```

Ошибка: {} создаёт пустой словарь, а не множество

```
empty = {}
```

```
print(type(empty))
```

Экспресс-опрос



Что такое хеш?



Хеш

Это числовое значение фиксированной длины, которое вычисляется из хешируемых объектов произвольной длины с помощью специальной хеш-функции.



Хешируемые объекты

Это типы данных, которые могут быть использованы в качестве ключей в хеш-таблицах или сохранены в структурах данных, использующих хеширование для сохранения и получения объектов

Экспресс-опрос

?

Назовите условия хешируемости

Условия хешируемости



Неизменность



Наличие метода `__hash__()`

Функция hash



Python предоставляет встроенную функцию `hash()`, которая возвращает хеш-значение для объектов, поддерживающих хеширование.

Примеры хешируемых объектов



Хеш-код для строки

```
print(hash("hello"))
```

```
print(hash("hello")) # Хеш-код одинаковый для одного значения
```

```
print(hash('1'))
```

Хеш-код для числа

```
print(hash(42))
```

```
print(hash(1))
```

```
print(hash(1.0))
```

Примеры хешируемых объектов



```
# Хеш-код для bool
```

```
print(hash(True))
```

```
# Хеш-код для кортежа
```

```
my_tuple = (1, 2, 3)
```

```
print(hash(my_tuple))
```

Примеры нехешируемых объектов

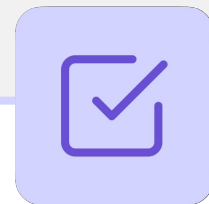


Списки являются изменяемыми объектами, поэтому они не поддерживают хеширование

```
my_list = [1, 2, 3]
```

```
# print(hash(my_list)) # Ошибка: TypeError: unhashable type: 'list'
```

Важно



Только хешируемые объекты могут выступать в качестве элементов в множествах, так как их хеш-код используется для быстрого доступа и проверки уникальности.

Экспресс-опрос

?

Для чего применяется хеширование?

Применение хеширования



Хеш-таблицы: В Python хеширование используется для реализации таких структур данных, как множества и словари.



Проверка целостности данных: Хеш-функции позволяют убедиться, что данные не были изменены.



Алгоритмы и структуры данных: Используется в алгоритмах поиска и шифрования данных.

Экспресс-опрос

?

Назовите принципы хранения и доступа

Принципы хранения и доступа



Хеширование элементов



Структура хранения



Коллизии



Неупорядоченность элементов



Поиск

Пример хранения элементов



```
my_set = {20, 10, 30, 40}
```

```
# Каждый элемент проходит через хеш-функцию
print(hash(20)) # Вывод хеш-кода для элемента 20
print(hash(10)) # Вывод хеш-кода для элемента 10
print(hash(30)) # Вывод хеш-кода для элемента 30
print(hash(40)) # Вывод хеш-кода для элемента 40
```

```
print(my_set)
```

Преобразование в множество



Функция `set()` используется для преобразования других итерируемых объектов (например, списков, строк, объекта `range`) в множество. Преобразование также удаляет дубликаты из коллекции, позволяя получить только уникальные элементы.

Примеры преобразования



Преобразование списка

```
numbers = [1, 2, 2, 3, 4, 4, 5]
unique_numbers = set(numbers)
print(unique_numbers)
```

Преобразование строки

```
text = "hello"
unique_chars = set(text)
print(unique_chars)
```

Преобразование объекта range

```
numbers = set(range(10))
print(numbers)
```

Экспресс-опрос

?

Назовите особенности преобразования

Особенности преобразования



Порядок элементов в результате преобразования не сохраняется, так как множества неупорядочены



В множестве остаются только уникальные элементы

Методы добавления и удаления элементов



Множества в Python предоставляют удобные методы для добавления и удаления элементов. Эти методы обеспечивают гибкость и безопасность при работе с уникальными значениями.

Экспресс-опрос

?

Какие методы добавления и удаления элементов для множеств вы знаете?

Метод	Описание	Синтаксис
<code>add()</code>	Добавляет один элемент в множество.	<code>my_set.add(element)</code>
<code>remove()</code>	Удаляет указанный элемент. Вызывает ошибку <code>KeyError</code> , если элемента нет.	<code>my_set.remove(element)</code>
<code>discard()</code>	Удаляет указанный элемент. Ошибки не возникает, если элемента нет.	<code>my_set.discard(element)</code>
<code>pop()</code>	Удаляет и возвращает случайный элемент. Вызывает ошибку <code>KeyError</code> , если множество пусто.	<code>removed_element = my_set.pop()</code>
<code>clear()</code>	Удаляет все элементы из множества, делая его пустым.	<code>my_set.clear()</code>

Пример: метод add()



```
# Метод add()

my_set = {1, 2, 3}

my_set.add(4)

my_set.add(3)

print(my_set)
```

Пример: метод remove()



```
# Метод remove()

my_set = {1, 2, 3}

my_set.remove(2)

print(my_set)

# my_set.remove(2) # Попытка повторного удаления вызовет ошибку KeyError
```

Пример: метод discard()



```
# Метод discard()

my_set = {1, 2, 3}

my_set.discard(2)

my_set.discard(4) # Ошибки не будет

print(my_set)
```

Пример: метод pop()



```
# Метод pop()

my_set = {1, 2}

removed_element = my_set.pop()

print(removed_element)

print(my_set) # Оставшиеся элементы

print(my_set.pop())

# print(my_set.pop()) # Вызовет ошибку KeyError, так как элементов больше нет.
```

Пример: метод clear()



```
# Метод clear()

my_set = {1, 2, 3}

my_set.clear()

print(my_set)
```

Пример: метод copy()



```
# Метод copy()

original_set = {1, 2, 3}

copied_set = original_set.copy()

# Проверим, что это отдельный объект

copied_set.add(4)

print(copied_set)

print(original_set)
```

Экспресс-опрос

?

Какие функции и операторы для множеств вы помните?

Функция len()



Пример

```
my_set = {1, 2, 3, 4}  
print(len(my_set))
```

Пояснение

возвращает количество элементов в
множестве

Функция min()



Пример

```
my_set = {10, 20, 5, 8}  
print(min(my_set))
```

Пояснение

Возвращает минимальный элемент в множестве

Функция max()



Пример

```
my_set = {10, 20, 5, 8}  
print(max(my_set))
```

Пояснение

Возвращает максимальный элемент в множестве

Функция sum()



Пример

```
my_set = {1, 2, 3, 4}  
print(sum(my_set))
```

Пояснение

Возвращает сумму всех элементов в множестве

Функция sorted()



Пример

```
my_set = {3, 1, 4, 2}  
print(sorted(my_set))
```

Пояснение

Возвращает отсортированный список
элементов множества

Оператор вхождения in



Пример

```
my_set = {1, 2, 3}  
print(2 in my_set)  
print(4 in my_set)
```

Пояснение

Проверяет, содержит ли множество
определённый элемент

Оператор вхождения not in



Пример

```
my_set = {1, 2, 3}  
print(4 not in my_set)
```

Пояснение

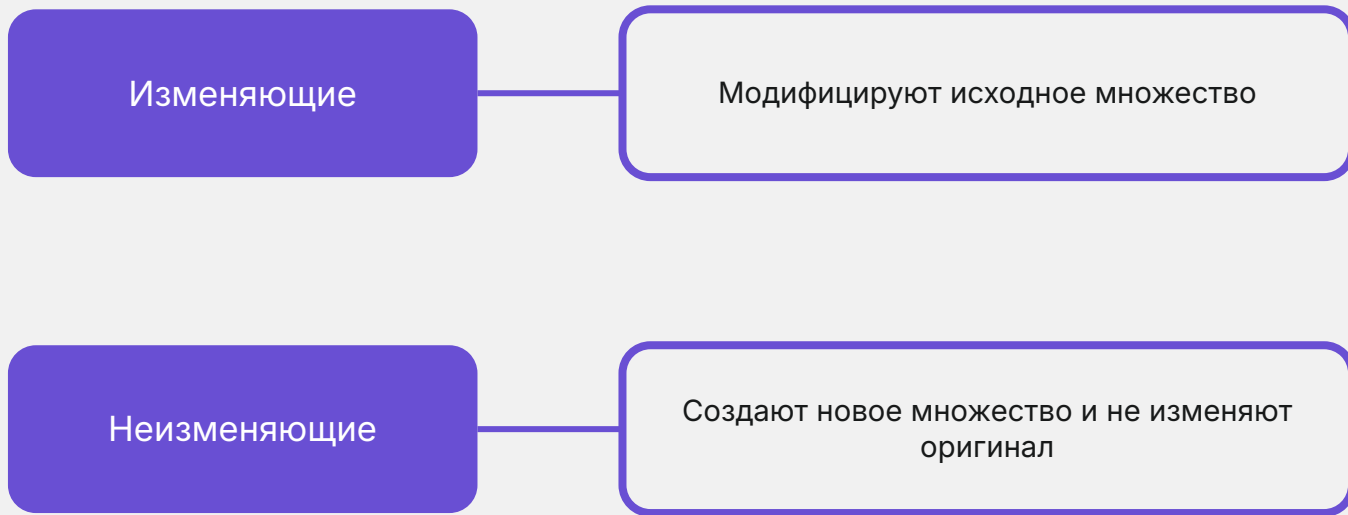
Проверяет, отсутствует ли элемент в множестве

Экспресс-опрос

?

Какими бывают методы операторов для множеств?

Методы



Объединение множеств



Описание

Объединяет элементы двух множеств, возвращая новое множество с уникальными элементами из обоих множеств

Детали

Оператор: |

Методы:

- Неизменяющий: `.union()` — создаёт новое множество.
- Изменяющий: `.update()` — изменяет исходное множество.

Пример объединения множеств



```
set1 = {1, 2, 3}
set2 = {3, 4, 5}
```

```
result1 = set1.union(set2) # Неизменяющий метод
result2 = set1 | set2      # Использование оператора
print(result1)
print(result2)
print(set1)
```

```
set1.update(set2) # Изменяющий метод
print(set1)
```

Пересечение множеств



Описание

Возвращает множество, содержащее элементы, которые присутствуют в обоих множествах

Детали

Оператор: &

Методы:

- Неизменяющий: `.intersection()`
- Изменяющий: `.intersection_update()`

Пример пересечения множеств



```
set1 = {1, 2, 3}
set2 = {2, 3, 4}
result1 = set1.intersection(set2) # Неизменяющий метод
result2 = set1 & set2 # Использование оператора
print(result1)
print(result2)
print(set1)

set1.intersection_update(set2) # Изменяющий метод
print(set1)
```

Разность множеств



Описание

Возвращает новое множество, содержащее элементы, которые есть в первом множестве, но отсутствуют во втором

Детали

Оператор: -

Методы:

- Неизменяющий: `.difference()`
- Изменяющий: `.difference_update()`

Пример разности множеств



```
set1 = {1, 2, 3}
set2 = {2, 3, 4}
result1 = set1.difference(set2) # Неизменяющий метод
result2 = set1 - set2 # Использование оператора
print(result1)
print(result2)
print(set1)

set1.difference_update(set2) # Изменяющий метод
print(set1)
```

Симметрическая разность



Описание

Возвращает новое множество, содержащее элементы, которые присутствуют в одном из множеств, но не в обоих одновременно

Детали

Оператор: ^

Методы:

- Неизменяющий: `.symmetric_difference()`
- Изменяющий:
`.symmetric_difference_update()`

Пример симметрической разности



```
set1 = {1, 2, 3}
set2 = {2, 3, 4}
result1 = set1.symmetric_difference(set2) # Неизменяющий метод
result2 = set1 ^ set2 # Использование оператора
print(result1)
print(result2)
print(set1)

set1.symmetric_difference_update(set2) # Изменяющий метод
print(set1)
```

Экспресс-опрос

?

Какие отношения могут быть у множеств?

Подмножество



Описание

Содержит ли одно множество все элементы другого множества

Детали

Оператор: $\leq$, $<$

Метод: `.issubset()`

Пример для подмножества



```
set1 = {1, 2, 3}
set2 = {1, 2}
set3 = {1, 2, 3}
print(set1 <= set2)
print(set2 <= set1)
print(set2 < set1)
print(set1.issubset(set3))
```

Надмножество



Описание

Содержит ли одно множество другое множество целиком

Детали

Оператор: $\geq$, $>$

Метод: `.issuperset()`

Пример для надмножества



```
set1 = {1, 2, 3}
set2 = {1, 2}
set3 = {1, 2, 3}
print(set1 >= set2)
print(set2 >= set1)
print(set1 > set3)
print(set1.issuperset(set3))
```

Равенство множеств



Описание

Содержат ли два множества одни и те же элементы

Детали

Оператор: ==

Метод: Нет метода

Пример для равенства множеств



```
set1 = {1, 2, 3}  
set2 = {3, 2, 1}  
print(set1 == set2)
```

Неравенство множеств



Описание

Содержат ли два множества разные элементы

Детали

Оператор: `!=`

Метод: Нет метода

Пример для неравенства множеств



```
set1 = {1, 2, 3}  
set2 = {3, 2, 1}  
print(set1 == set2)
```

Отсутствие пересечений



Описание

Содержат ли два множества разные элементы

Детали

Оператор: Нет

Метод: `.isdisjoint()`

Пример для отсутствия пересечений



```
set1 = {1, 2}  
set2 = {3, 4}  
set3 = {2, 3}  
print(set1.isdisjoint(set2))  
print(set1.isdisjoint(set3))
```

Экспресс-опрос

?

В чем особенность цикла по множеству?

Цикл по множеству



В Python можно использовать цикл `for` для перебора элементов множества. Поскольку множество — это неупорядоченная коллекция уникальных элементов, порядок их обхода в цикле не гарантируется и может отличаться при каждом запуске программы

Пример цикла по множеству



```
my_set = {10, 20, 30, 40, 50}

for item in my_set:
    print(item)
```

Экспресс-опрос

?

Что такое Set comprehension?

Set comprehension



Это способ создания множества на основе другой последовательности или итерируемого объекта с применением условий и выражений. Синтаксис полностью аналогичен List comprehension, за исключением фигурных скобок вместо квадратных.

Синтаксис



```
new_set = {expression for item in iterable}
```

Пример



```
# Создание множества с числами
numbers = [1, 2, 2, 4, 7, 8, 8, 10]
even_numbers_set = {num for num in numbers if num % 2 == 0}
print(even_numbers_set)
```

Экспресс-опрос

?

Что такое frozenset?



frozenset

Это неизменяемый аналог обычного множества. В отличие от множества, элементы frozenset не могут быть добавлены, удалены или изменены после создания.

frozenset



`frozenset` создаётся с помощью одноимённой функции, в которую передается итерируемый объект.

frozenset пример



```
immutable_set = frozenset([1, 2, 3, 4, 5])  
  
print(immutable_set)  
  
immutable_from_range = frozenset(range(10))  
  
print(immutable_from_range)
```

Хешируемость frozenset



`frozenset` является неизменяемым и хешируемым объектом. Это позволяет использовать его в качестве элемента множества, в отличие от обычного множества `set`, которое не может быть элементом другого множества из-за своей изменяемости.

Пример



```
# Создание frozenset
```

```
frozen_set1 = frozenset([1, 2, 3])
```

```
frozen_set2 = frozenset([4, 5, 6])
```

```
# Создание множества, содержащего frozenset
```

```
set_of_frozensets = {frozen_set1, frozen_set2}
```

```
print(set_of_frozensets)
```

Сравнение set и frozenset



Это оба типа множеств, которые поддерживают уникальные элементы и позволяют выполнять множество операций, таких как объединение, пересечение и разность. Однако между ними есть важные различия, которые влияют на их применение.

Экспресс-опрос

?

Какие отличия между `set` и `frozenset` вы помните?

Характеристика	set	frozenset
Изменяемость	Изменяемый: поддерживает методы <code>add()</code> , <code>remove()</code> , <code>discard()</code> , <code>pop()</code> , <code>clear()</code> .	Неизменяемый: методы изменения отсутствуют. Поддерживаются только методы, возвращающие новый объект.
Хешируемость	Не является хешируемым, поэтому не может быть элементом других множеств или ключом словаря.	Хешируемый: может использоваться как элемент множества или ключ словаря.

Экспресс-опрос

?

Что такое словарь в Python?



Словарь (ассоциативный массив)

Это структура данных, представляющая собой неупорядоченную изменяемую коллекцию пар "ключ-значение"

Создание словаря



Синтаксис

```
my_dict = {
    key1: value1,
    key2: value2,
    ...
}
```

Пояснение

- `key` — уникальный объект (ключ), по которому можно получить соответствующее значение.
- `value` — данные (значение), которые хранятся по ключу.

Создание словаря с использованием фигурных скобок



Пример

```
person = {"name": "Alice", "age": 30,  
          "city": "New York"}  
  
print(person)
```

Пояснение

Наиболее распространённый способ создания словаря, в котором указываются ключи и соответствующие им значения

Создание пустого словаря



Пример

```
empty_dict = {}  
  
print(empty_dict)
```

```
empty_dict = dict()  
  
print(empty_dict)
```

Пояснение

Создать пустой словарь для последующего добавления элементов можно с помощью фигурных скобок или функции `dict()`.

Экспресс-опрос

?

Назовите особенности хранения словаря в памяти

Хранение словаря в памяти

Хеширование ключей

Хеш-функция вычисляет уникальное хеш-значение, что позволяет словарям быстро находить значение по ключу

Хеш-таблица

Хранение пар "ключ-значение". При одинаковом хеш-значении (коллизии), используется метод цепочек для разрешения конфликта

Занимаемая память

Размер таблицы превышает размер ее содержимого. При добавлении новых элементов возможно рехеширование

Порядок элементов

Начиная с Python 3.7, словари сохраняют порядок вставки элементов. Использование упорядоченных хеш-таблиц

Хранение словаря в памяти



Преимущества

- Быстрый доступ
- Гибкость
- Уникальные ключи

Ограничения

- Память
- Изменяемость ключей

Особенности словарей



Ключи должны быть уникальными и хешируемыми



Гибкость значений



Быстрый доступ к значениям



Сохранение порядка элементов (начиная с Python 3.7)



Изменяемость значений

Экспресс-опрос

?

Какие особенности хеширования вы помните?

Особенности хеширования



В Python числа 1, 1.0 и значение True обладают одинаковым хешем. При использовании 1, 1.0 и True в качестве ключей в одном словаре или элементах множества, они будут рассматриваться как один и тот же ключ или элемент.

Особенности хеширования

Проверка

```
print(hash(1))  
print(hash(1.0))  
print(hash(True))
```

Пример для словаря

```
my_dict = {1.0: "float", 1: "integer",  
True: "boolean"}  
print(my_dict)
```

Особенности хеширования

Пример для множества

```
my_set = {True, 1, 1.0}  
print(my_set)
```

Пояснения

Множества будут содержать только один из таких элементов

Экспресс-опрос

?

Какие особенности преобразования в словарь вы помните?

Особенности преобразования



Ключи в словаре должны быть хешируемыми и уникальными



При преобразовании коллекций с дублирующимися ключами будет сохранено только последнее значение



Если структура данных не совместима с форматом "ключ-значение", преобразование в словарь приведёт к ошибке

Преобразование именованных аргументов в словарь

Пример

```
# Словарь с использованием функции dict()

person = dict(name="Bob", age=25,
               city="London")

print(person)
```

Пояснения

Функция `dict()` позволяет создавать словарь из пар ключ-значение, передаваемых как именованные аргументы

Преобразование других коллекций в словарь



Функция `dict()` может быть использована для преобразования последовательности пар (например, списка кортежей) в словарь. Важно чтобы вложенные коллекции состояли ровно из двух элементов и имели неизменяемый элемент на нулевом индексе

Примеры

Список кортежей

```
pairs = [("name", "Charlie"), ("age", 35),  
         ("city", "Paris")]
```

```
person = dict(pairs)
```

```
print(person)
```

Список списков

```
pairs = [("name", "Charlie"), ["age", 35],  
         ["city", "Paris"]]
```

```
person = dict(pairs)
```

```
print(person)
```

Примеры

Несоответствующее количество элементов

```
not_pairs = [("name", "Charlie"), ["age",
35], ["city", "Paris", "Berlin"]]
```

```
person = dict(not_pairs) # Вызовет ошибку
```

```
print(person)
```

Нехешируемый ключ

```
pairs = ([["name", "surname"], "Charlie"),
["age", 35], ["city", "Paris"]]
```

```
person = dict(pairs) # Вызовет ошибку
```

```
print(person)
```

Доступ к значениям по ключам

Синтаксис

```
variable = dictionary[key]
```

Пояснения

- `dictionary` — объект словаря, из которого вы хотите получить значение
- `key` — ключ, по которому будет извлечено связанное значение
- `variable` — переменная, в которой можно сохранить значение, связанное с указанным ключом

Доступ к значениям по ключам

Особенности

- Если ключ существует, будет возвращено соответствующее значение.
- Если ключ отсутствует, возникает ошибка `KeyError`.

Пример

```
# Получение значения по ключу

my_dict = {"name": "Alice", "age": 30}

print(my_dict["name"])

age = my_dict["age"]

print(age)

# print(my_dict["city"]) # Вызовет ошибку
```

Экспресс-опрос

?

Какой оператор используется для проверки, существует ли указанный ключ в словаре?

Оператор in



Используется для проверки, существует ли указанный ключ в словаре. Он часто используется для проверки наличия ключа перед доступом к значению, чтобы избежать ошибки `KeyError`.

Пример



```
my_dict = {"name": "Alice", "age": 30}

if "name" in my_dict:
    print(my_dict["name"]) # Выведет значение по ключу 'name'

if "city" in my_dict:
    print(my_dict["city"]) # Не выполняется, так как ключ 'city' отсутствует
```

Цикл по словарю



По словарям можно итерироваться с помощью цикла `for`. По умолчанию цикл `for` перебирает ключи словаря, с помощью которых можно получить значения.

Пример



```
my_dict = {"name": "Alice", "age": 30, "city": "New York"}  
  
for key in my_dict:  
    print(f"Ключ={key}, значение={my_dict[key]}")
```

Добавление и обновление данных



Словари позволяют быстро добавлять новые пары "ключ-значение" или обновлять существующие значения по ключу.

Добавление новой пары "ключ-значение"



```
my_dict = {"name": "Alice", "age": 30}
my_dict["city"] = "New York" # Добавление нового элемента
print(my_dict)
```

Обновление значения по существующему ключу



```
my_dict = {"name": "Alice", "age": 30}
my_dict["age"] = 31 # Обновление значения по ключу "age"
print(my_dict)
```

Экспресс-опрос

?

Что делает метод `update()`?

Метод update()



Метод `update()` позволяет добавлять несколько пар "ключ-значение" или обновлять значения существующих ключей. Можно передать другой словарь или другую последовательность пар ключ-значение.

Метод update()



Обновление значений и добавление новых ключей

```
my_dict = {"name": "Alice", "age": 30}

my_dict.update({"age": 32, "country": "USA"})

print(my_dict)
```

Обновление с использованием списка кортежей

```
my_dict.update([("name", "Bob"), ("email", "bob@example.com")])

print(my_dict)
```

Обновление с использованием именованных аргументов

```
my_dict.update(city="New York", orders=[])

print(my_dict)
```

Особенности добавления и обновления



- Если ключ уже существует, его значение будет обновлено.
- Если ключ отсутствует, он будет добавлен в словарь.

Экспресс-опрос

?

Какие методы удаления данных вы помните?



Оператор del

Этот оператор удаляет элемент с указанным ключом из словаря. Если ключ отсутствует, возникает ошибка `KeyError`.

Оператор del. Пример



```
my_dict = {"name": "Alice", "age": 30, "city": "New York"}  
  
del my_dict["age"]  
  
print(my_dict)  
  
# del my_dict["email"] # Вызовет ошибку
```



Метод `clear()`

Этот метод удаляет все элементы из словаря, оставляя пустой словарь.

Метод clear(). Пример



```
my_dict = {"name": "Alice", "age": 30}  
my_dict.clear()  
print(my_dict)
```



Метод pop()

Этот метод удаляет элемент по указанному ключу и возвращает его значение. Если ключ отсутствует и значение по умолчанию не указано, возникает ошибка `KeyError`.

Метод pop(). Пример



```
my_dict = {"name": "Alice", "age": 30}
age = my_dict.pop("age")
print(age)
print(my_dict)

# my_dict.pop("email") # Вызовет ошибку
```



Метод `popitem()`

Удаляет и возвращает последнюю добавленную пару (начиная с Python 3.7) или случайную пару. Если словарь пуст, возникает ошибка `KeyError`.

Метод popitem(). Пример



```
my_dict = {"name": "Alice", "age": 30}

last_item = my_dict.popitem()

print(last_item)

print(my_dict)
```



ВОПРОСЫ





ЗАДАНИЯ ДЛЯ ЗАКРЕПЛЕНИЯ





Выберите верный вариант ответа

1.1 Какие числа будут выведены при выполнении следующего кода?

```
elements = [1, 2, 3, 4, 3, 2, 1]
```

```
unique_elements = set(elements)
```

```
print(unique_elements)
```

- a. {1, 2, 3, 4, 3, 2, 1}
- b. {1, 2, 3, 4}
- c. [1, 2, 3, 4]
- d. [1, 2, 3, 4, 3, 2, 1]



Выберите верный вариант ответа

1.1 Какие числа будут выведены при выполнении следующего кода?

```
elements = [1, 2, 3, 4, 3, 2, 1]
```

```
unique_elements = set(elements)
```

```
print(unique_elements)
```

- a. {1, 2, 3, 4, 3, 2, 1}
- b. {1, 2, 3, 4}**
- c. [1, 2, 3, 4]
- d. [1, 2, 3, 4, 3, 2, 1]



Выберите верный вариант ответа

1.2 Что произойдёт при выполнении следующего кода?

```
data = {10, 20, (30, 40)}
```

```
print(hash((30, 40)))
```

- a. Ошибка: TypeError
- b. Выведено числовое значение, представляющее хеш-код кортежа
- c. Выведено None
- d. Выведено (30, 40)



Выберите верный вариант ответа

1.2 Что произойдёт при выполнении следующего кода?

```
data = {10, 20, (30, 40)}
```

```
print(hash((30, 40)))
```

- a. Ошибка: TypeError
- b. Выведено числовое значение, представляющее хеш-код кортежа
- c. Выведено None
- d. Выведено (30, 40)



Выберите верный вариант ответа

1.3 Почему элементы множества в Python должны быть хешируемыми?

- a. Чтобы элементы можно было сравнивать друг с другом
- b. Чтобы элементы можно было упорядочить
- c. Чтобы элементы можно было добавлять и удалять
- d. Чтобы обеспечить быстрый доступ и проверку уникальности



Выберите верный вариант ответа

1.3 Почему элементы множества в Python должны быть хешируемыми?

- a. Чтобы элементы можно было сравнивать друг с другом
- b. Чтобы элементы можно было упорядочить
- c. Чтобы элементы можно было добавлять и удалять
- d. Чтобы обеспечить быстрый доступ и проверку уникальности



Выберите верный вариант ответа

1.4 Какой результат будет выведен при выполнении следующего кода?

```
data = {1, 2, 3}
```

```
data.add((4, 5))
```

```
print(data)
```

- a. {1, 2, 3, (4, 5)}
- b. {1, 2, 3, (4), (5)}
- c. {1, 2, 3, 4, 5}
- d. Ошибка: TypeError



Выберите верный вариант ответа

1.4 Какой результат будет выведен при выполнении следующего кода?

```
data = {1, 2, 3}
```

```
data.add((4, 5))
```

```
print(data)
```

- a. {1, 2, 3, (4, 5)}
- b. {1, 2, 3, (4), (5)}
- c. {1, 2, 3, 4, 5}
- d. Ошибка: TypeError



Выберите верный вариант ответа

1.5 Что произойдёт, если попытаться добавить список в множество?

- a. Список будет добавлен в множество отдельными элементами
- b. Список будет преобразован в кортеж и добавлен
- c. Возникнет ошибка: `TypeError`
- d. Список будет проигнорирован



Выберите верный вариант ответа

1.5 Что произойдёт, если попытаться добавить список в множество?

- a. Список будет добавлен в множество отдельными элементами
- b. Список будет преобразован в кортеж и добавлен
- c. Возникнет ошибка: `TypeError`
- d. Список будет проигнорирован



Выберите верный вариант ответа

1.6 Как Python разрешает коллизии при хранении элементов в множестве?

- a. Путём удаления дублирующихся элементов
- b. Путём сортировки элементов в порядке их добавления
- c. С помощью метода цепочек (связных списков)
- d. С помощью метода линейного поиска



Выберите верный вариант ответа

1.6 Как Python разрешает коллизии при хранении элементов в множестве?

- a. Путём удаления дублирующихся элементов
- b. Путём сортировки элементов в порядке их добавления
- c. С помощью метода цепочек (связных списков)**
- d. С помощью метода линейного поиска



Выберите верный вариант ответа

1.7 Какая структура данных лежит в основе множества в Python?

- a. Двоичное дерево
- b. Хеш-таблица**
- c. Стеки
- d. Очередь

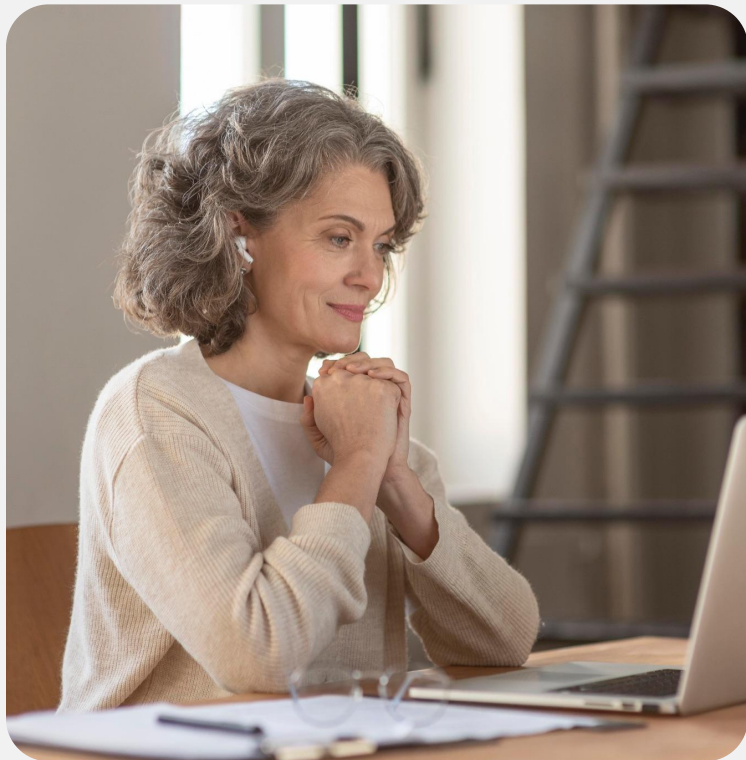


Выберите верный вариант ответа

1.8 Какой результат будет выведен при выполнении следующего кода?

```
my_set = {"10", "20", "30"}  
print(my_set.pop())
```

- a. 10
- b. 30
- c. Случайный элемент из множества
- d. Ошибка

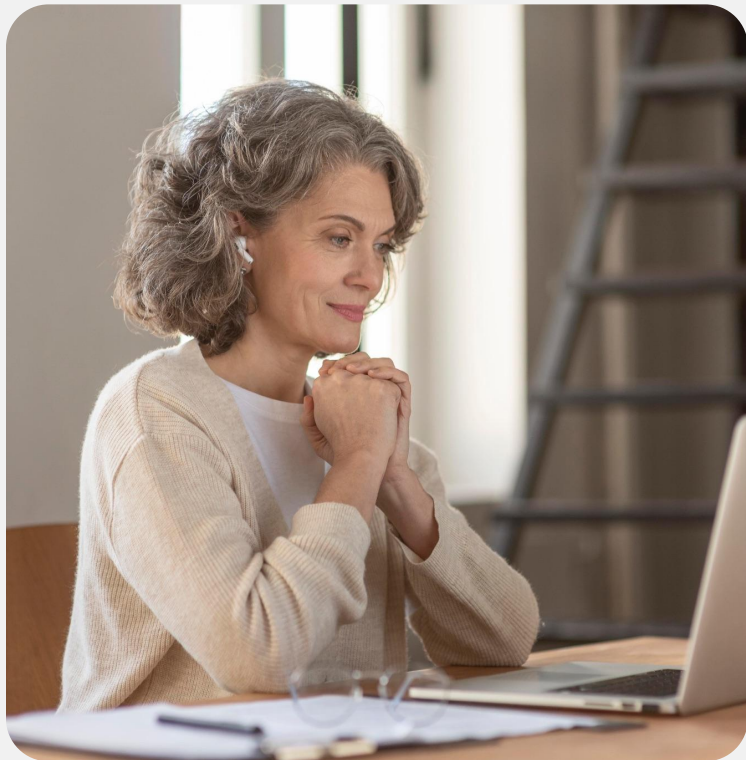


Выберите верный вариант ответа

1.8 Какой результат будет выведен при выполнении следующего кода?

```
my_set = {"10", "20", "30"}
print(my_set.pop())
```

- a. 10
- b. 30
- c. Случайный элемент из множества
- d. Ошибка



Выберите верный вариант ответа

1.9 Какой оператор используется для выполнения симметрической разности множеств?

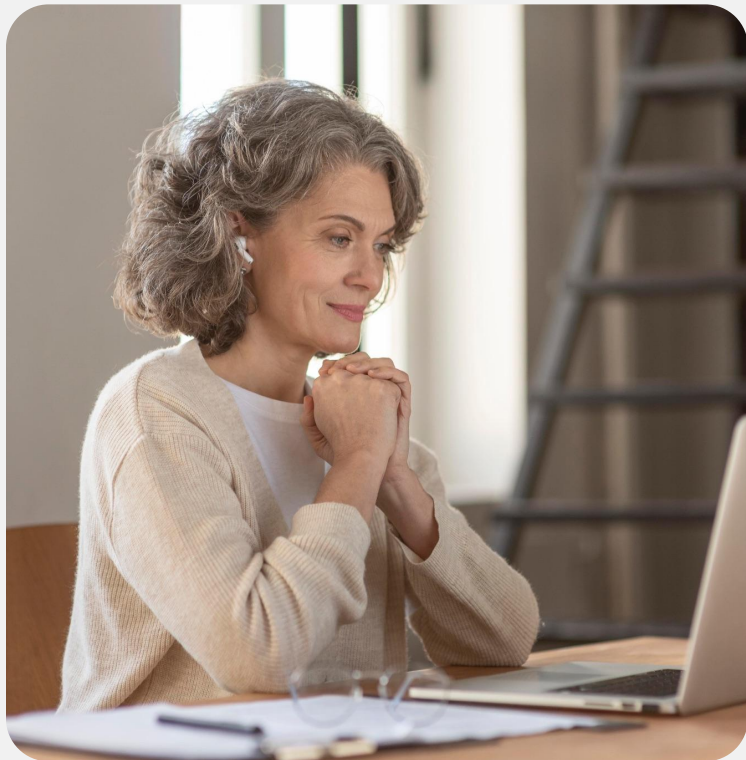
- a. $\&$
- b. $\wedge$
- c. $|$
- d. $-$



Выберите верный вариант ответа

1.9 Какой оператор используется для выполнения симметрической разности множеств?

- a. $\&$
- b. $\wedge$
- c. $|$
- d. $-$



Выберите верный вариант ответа

1.10 Какой результат будет выведен при выполнении следующего кода?

```
set1 = {1, 2, 3}
set2 = {3, 4, 5}
set1.symmetric_difference_update(set2)
print(set1)
```

- a. {1, 2, 3, 4, 5}
- b. {3, 4, 5}
- c. {1, 2, 4, 5}
- d. {1, 2, 3}



Выберите верный вариант ответа

1.10 Какой результат будет выведен при выполнении следующего кода?

```
set1 = {1, 2, 3}
set2 = {3, 4, 5}
set1.symmetric_difference_update(set2)
print(set1)
```

- a. {1, 2, 3, 4, 5}
- b. {3, 4, 5}
- c. {1, 2, 4, 5}**
- d. {1, 2, 3}



ВОПРОСЫ





Выберите верный вариант ответа

2.1 Какой результат будет выведен при выполнении следующего кода?

```
frozen_set = frozenset([1, 2, 3, 4])
frozen_set.append(5)
```

- a. {1, 2, 3, 4, 5}
- b. {1, 2, 3, 4}
- c. [1, 2, 3, 4, 5]
- d. Ошибка



Выберите верный вариант ответа

2.1 Какой результат будет выведен при выполнении следующего кода?

```
frozen_set = frozenset([1, 2, 3, 4])
frozen_set.append(5)
```

- a. {1, 2, 3, 4, 5}
- b. {1, 2, 3, 4}
- c. [1, 2, 3, 4, 5]
- d. Ошибка



Выберите верный вариант ответа

2.2 Какие утверждения верны относительно использования frozenset?

- a. frozenset может быть элементом множества set.
- b. frozenset можно использовать в качестве ключа словаря.
- c. frozenset является изменяемым объектом.
- d. frozenset поддерживает метод `remove()`.



Выберите верный вариант ответа

2.2 Какие утверждения верны относительно использования frozenset?

- a. frozenset может быть элементом множества set.
- b. frozenset можно использовать в качестве ключа словаря.
- c. frozenset является изменяемым объектом.
- d. frozenset поддерживает метод remove().



Выберите верный вариант ответа

2.3 Как создается пустой словарь?

- a. `empty_dict = []`
- b. `empty_dict = {}`
- c. `empty_dict = set()`
- d. `empty_dict = dict()`



Выберите верный вариант ответа

2.3 Как создается пустой словарь?

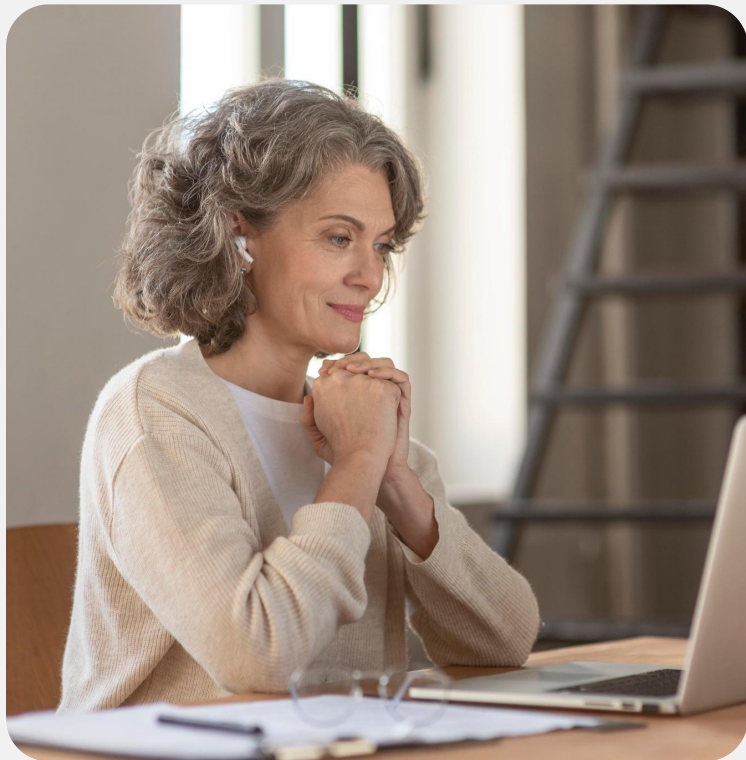
- a. `empty_dict = []`
- b. `empty_dict = {}`
- c. `empty_dict = set()`
- d. `empty_dict = dict()`



Выберите верный вариант ответа

2.4 Какие типы данных можно использовать в качестве ключа словаря?

- a. int
- b. float
- c. str
- d. bool
- e. lst
- f. tuple
- g. set
- h. frozenset
- i. dict



Выберите верный вариант ответа

2.4 Какие типы данных можно использовать в качестве ключа словаря?

- a. int
- b. float
- c. str
- d. bool
- e. lst
- f. tuple
- g. set
- h. frozenset
- i. dict



Выберите верный вариант ответа

2.5 Что произойдет, если добавить в словарь элемент с уже существующим ключом?

- a. Возникнет ошибка
- b. Новое значение будет добавлено в список после старого
- c. Значение существующего ключа будет заменено на новое
- d. Ключ будет удален из словаря



Выберите верный вариант ответа

2.5 Что произойдет, если добавить в словарь элемент с уже существующим ключом?

- a. Возникнет ошибка
- b. Новое значение будет добавлено в список после старого
- c. Значение существующего ключа будет заменено на новое
- d. Ключ будет удален из словаря



Выберите верный вариант ответа

2.6 Какой результат будет выведен при выполнении следующего кода?

```
person = {"name": "Alice", "age": 30}
print(person["name"])
```

- a. Alice
- b. "Name"
- c. 30
- d. Ошибка



Выберите верный вариант ответа

2.6 Какой результат будет выведен при выполнении следующего кода?

```
person = {"name": "Alice", "age": 30}
print(person["name"])
```

- a. Alice
- b. "Name"
- c. 30
- d. Ошибка



Выберите верный вариант ответа

2.7 Какой результат будет выведен при выполнении следующего кода?

```
my_dict = {1: "one", 1.0: "float one", True:
"boolean one"}
print(my_dict)
```

- a. {1: "one", 1.0: "float one", True: "boolean one"}
- b. {1: "boolean one"}
- c. {1: "one"}
- d. {True: "boolean one"}



Выберите верный вариант ответа

2.7 Какой результат будет выведен при выполнении следующего кода?

```
my_dict = {1: "one", 1.0: "float one", True:
"boolean one"}
print(my_dict)
```

- a. {1: "one", 1.0: "float one", True: "boolean one"}
- b. {1: "boolean one"}**
- c. {1: "one"}
- d. {True: "boolean one"}



Выберите верный вариант ответа

2.8 Какая из следующих структур данных используется для хранения словаря?

- a. Список
- b. Хеш-таблица
- c. Кортеж
- d. Множество



Выберите верный вариант ответа

2.8 Какая из следующих структур данных используется для хранения словаря?

- a. Список
- b. Хеш-таблица**
- c. Кортеж
- d. Множество



Выберите верный вариант ответа

2.9 Какой результат будет выведен при выполнении следующего кода?

```
person = dict(name="Alice", age=25, city="Paris")
print(person)
```

- a. `{"name": "Alice", "age": 25, "city": "Paris"}`
- b. `{("name", "Alice"), ("age", 25), ("city", "Paris")}`
- c. `["Alice", 25, "Paris"]`
- d. `dict(name="Alice", age=25, city="Paris")`



Выберите верный вариант ответа

2.9 Какой результат будет выведен при выполнении следующего кода?

```
person = dict(name="Alice", age=25, city="Paris")
print(person)
```

- a. `{"name": "Alice", "age": 25, "city": "Paris"}`
- b. `{("name", "Alice"), ("age", 25), ("city", "Paris")}`
- c. `["Alice", 25, "Paris"]`
- d. `dict(name="Alice", age=25, city="Paris")`



Выберите верный вариант ответа

2.10 Что произойдет, если ключ отсутствует в словаре при попытке доступа через квадратные скобки?

```
my_dict = {"name": "Alice", "age": 30}
print(my_dict["city"])
```

- a. Выведет None
- b. Выведет пустую строку
- c. Выведет пустой словарь
- d. Вызовет ошибку KeyError



Выберите верный вариант ответа

2.10 Что произойдет, если ключ отсутствует в словаре при попытке доступа через квадратные скобки?

```
my_dict = {"name": "Alice", "age": 30}
print(my_dict["city"])
```

- a. Выведет None
- b. Выведет пустую строку
- c. Выведет пустой словарь
- d. Вызовет ошибку `KeyError`



Выберите верный вариант ответа

2.11 Что произойдет при попытке создать словарь с нехешируемым ключом?

```
pairs = [(["name", "surname"], "Charlie"), ["age", 35], ["city", "Paris"]]
person = dict(pairs)
print(person)
```

- a. Словарь будет создан успешно
- b. Будет вызвана ошибка TypeError
- c. Нехешируемый ключ будет пропущен
- d. Каждый элемент списка будет отдельным ключом



Выберите верный вариант ответа

2.11 Что произойдет при попытке создать словарь с нехешируемым ключом?

```
pairs = [(["name", "surname"], "Charlie"), ["age", 35], ["city", "Paris"]]
person = dict(pairs)
print(person)
```

- a. Словарь будет создан успешно
- b. Будет вызвана ошибка `TypeError`
- c. Нехешируемый ключ будет пропущен
- d. Каждый элемент списка будет отдельным ключом



ВОПРОСЫ





ПРАКТИЧЕСКАЯ РАБОТА



1. Удаление повторяющихся букв

Напишите программу, которая принимает строку текста и возвращает новую строку, содержащую только уникальные символы, оставленные в порядке их первого появления.

Данные:

```
text = "banana"
```

Пример вывода:

Уникальные символы: ban

2. Подсчёт гласных в строках

Задача:

Создайте словарь, где ключами будут строки из списка, а значениями — количество гласных букв в этих строках.

Данные:

```
strings = ["привет", "мир", "python", "задача"]  
vowels = "аеёиоуыэюя"
```

Пример вывода:

```
{'hello': 2, 'world': 1, 'programming': 3, 'task': 1}
```



РАЗБОР ДОМАШНЕГО ЗАДАНИЯ



Домашнее задание

Проверка на подмножество

Напишите программу, которая проверяет, содержит ли первое множество все элементы второго множества. Реализуйте решение несколькими способами. Решите одним из способов не используя возможности множеств.

Данные:

```
set1 = {1, 2, 3, 4}  
set2 = {2, 3}
```

Пример вывода:

```
True
```

Домашнее задание

Зеркальное подмножество

Напишите программу, которая проверяет, являются ли элементы одного из множеств подмножеством другого. В случае положительного ответа возвращает разницу между двумя множествами. Проверить необходимо в обе стороны.

Данные:

```
set1 = {2, 3, 4, 5, 6}  
set2 = {4, 5}
```

Пример вывода:

```
Подмножество: True  
Разница: {2, 3, 6}
```



ВОПРОСЫ



Домашнее задание

1. Не уникальные числа

Напишите программу, которая находит все числа, встречающиеся более одного раза в списке, и выводит их в порядке убывания.

Данные:

```
numbers = [4, 7, 3, 7, 8, 3, 4, 2, 7, 3, 8, 4]
```

Пример вывода:

Числа, встречающиеся более одного раза: [7, 4, 3, 8]

Домашнее задание

2. Проверка подмножества

Напишите программу, которая проверяет, является ли один словарь подмножеством другого (т.е. все его пары "ключ-значение" содержатся в другом словаре).

Данные:

```
dict1 = {"a": 1, "b": 2}  
dict2 = {"a": 1, "b": 2, "c": 3}
```

Пример вывода:

Первый словарь является подмножеством второго.



ВОПРОСЫ



Заключение

